Figure 8.2 Dependency diagram using DI . `RegisterUser` indirectly depends on all the other classes but doesn't need to know how to create them. The `RegisterUser` handler declares that it requires `EmailSender`, and the container provides it.

The term *DI* is often used interchangeably with *IoC*. But DI is a specific version of the more general principle of IoC. In the context of ASP.NET Core,

- Without IoC, you'd write the code to listen for requests, check which handler to invoke, and then invoke it. With IoC, the control flow is the other way around. You register your handlers with the framework, but it's up to the framework to invoke your handler. Your handler is still responsible for creating its dependencies.
- DI takes IoC one step further. As well as invoking your handler, with DI, the framework creates all your handler's dependencies.

So when you use dependency injection, your `RegisterUser` handler is no longer responsible for controlling how to create an `EmailSender` instance. Instead, the framework provides an `EmailSender` to the handler directly.

NOTE Many DI containers are available for .NET, including Autofac, Lamar, Unity, Ninject, and Simple Injector, and the list goes on! In chapter 31 you'll see how to replace the default ASP.NET Core container with one of these alternatives.

The advantage of adopting this pattern becomes apparent when you see how much it simplifies using dependencies. Listing 8.4 shows how the `RegisterUser` handler would look if you used DI to create